



## Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

# Laboratorio de Computación Salas A y B

*Profesor(a):* Carlos Chaves Mercado

*Asignatura:* Fundamentos de programación

*Grupo:* 8

*No de Práctica(s):* Práctica de estudio 06: Entorno y fundamentos del lenguaje C

*Integrante(s):* Nolasco Ramírez Sofía

Díaz Rodríguez José Pablo

Cruz Gómez Lizette Jacqueline

Macay Martínez David

*No. de lista o  
brigada:* Mesa 6

*Semestre:* 2026-2

*Fecha de entrega:* 27 de marzo de 2026

*Observaciones:*

**CALIFICACIÓN:** \_\_\_\_\_

**Objetivo** : El alumnado elaborará programas en lenguaje C utilizando las instrucciones de control de tipo secuencia, para realizar la declaración de variables de diferentes tipos de datos, así como efectuar llamadas a funciones externas de entrada y salida para asignar y mostrar valores de variables y expresiones. Actividades:

- Crear un archivo de texto (utilizando algún editor) y escribir un programa en lenguaje C que contenga variables de diferentes tipos, asignación de valores (por lectura desde la entrada estándar o asignación directa) y escritura del valor de las variables en la salida estándar
- Compilar un código fuente y ejecutarlo.
- Modificar y actualizar un programa usando un editor.
- Elaborar expresiones relacionales/lógicas en un programa en C y mostrar el resultado de su evaluación.

## Introducción

El lenguaje de programación C es uno de los más utilizados y fundamentales en el mundo de la informática. Fue desarrollado en los años 70 por Dennis Ritchie en los laboratorios Bell con el propósito de mejorar la eficiencia y portabilidad de los sistemas operativos, en particular, Unix. Su importancia radica en que es un lenguaje de bajo nivel, pero con características de alto nivel, lo que permite un control preciso del hardware sin sacrificar la legibilidad del código.

A través de estas actividades, como estudiantes desarrollaremos habilidades esenciales en C, como la declaración de variables, el uso de estructuras de control secuenciales y la manipulación de funciones de entrada y salida. La práctica con la compilación y ejecución de programas reforzará nuestra comprensión de cómo se transforma el código fuente en instrucciones ejecutables, una base fundamental para avanzar en el mundo de la programación

1. Con ayuda del manual de la Práctica 06, seguir las instrucciones que dará la profesora para aprender a utilizar el editor de texto Visual Interface (vi) creando un archivo de texto plano. Para el desarrollo del reporte será necesario documentar los pasos que se vayan siguiendo y tomar las capturas de pantalla correspondientes.
2. Al igual que en el punto 1, deberán seguir las instrucciones proporcionadas por la profesora para utilizar el editor de texto GNU nano creando y editando un archivo de texto plano. Documentar el proceso para la entrega del reporte.
3. Utilizando el editor que prefieran (vi o nano) codificar, compilar y ejecutar los

problemas propuestos en el manual de la práctica del 1 al 4. Tomar evidencias como se indique.

## Desarrollo

La práctica 6 se desarrolló mediante la elaboración de un programa en lenguaje C en el cual se integraron los conceptos básicos del entorno de programación, la estructura del lenguaje y el uso de instrucciones de entrada, salida y control.

En primer lugar, se trabajó con la estructura general de un programa en C, identificando sus componentes principales como la inclusión de bibliotecas mediante la directiva:

```
#include <stdio.h>
```

así como la función principal main() y el uso de llaves para delimitar bloques de instrucciones. Esto permitió comprender cómo inicia y termina la ejecución de un programa.

Posteriormente, se realizó la declaración de variables utilizando distintos tipos de datos: int para la edad, float para la altura y char para almacenar la inicial del nombre, como se muestra a continuación:

```
int                edad,                dobleEdad;
float              altura;
char inicial;
```

Esto permitió reconocer la importancia de seleccionar correctamente el tipo de dato según la información que se desea manejar.

En la etapa de entrada de datos, se utilizó la función scanf para capturar valores desde el teclado:

```
scanf("%d",                                &edad);
scanf("%f",                                &altura);
scanf(" %c", &inicial);
```

Se emplearon los especificadores de formato correspondientes (%d, %f, %c) junto con el operador de dirección (&) para almacenar correctamente los datos en memoria. Durante este proceso se identificaron advertencias relacionadas con el uso incorrecto de formatos, lo que permitió comprender la importancia de la correspondencia entre el tipo de variable y el especificador utilizado.

Para la salida de información, se utilizó la función printf, mostrando los datos ingresados por el usuario con distintos formatos, como por ejemplo:

```
printf("\nAltura: %.2f", altura);
```

incluyendo el uso de precisión en números flotantes. Esto permitió visualizar cómo se pueden presentar los resultados de manera clara y organizada.

Además, se implementaron estructuras de control condicionales mediante instrucciones if y else, las cuales permitieron evaluar condiciones como la mayoría de edad:

```
if (edad >= 18) {
    printf("Eres mayor de edad\n");
} else {
    printf("Eres menor de edad\n");
}
```

y la clasificación de la altura (mayor o menor a 1.70 m):

```
if (altura > 1.70) {
    printf("Tu altura es alta\n");
} else {
    printf("Tu altura es promedio o baja\n");
}
```

Esto ayudó a comprender cómo un programa puede tomar decisiones y ejecutar diferentes acciones dependiendo de los valores de entrada.

También se realizó una operación aritmética, calculando el doble de la edad del usuario:

```
dobleEdad = edad * 2;
```

y almacenando el resultado en una nueva variable. Esto reforzó el uso de operadores básicos y la manipulación de datos dentro del programa.

Durante el desarrollo, se llevó a cabo el proceso de compilación, en el cual se identificaron errores de sintaxis, como paréntesis faltantes y variables no declaradas, así como advertencias en el uso de funciones. Estos errores fueron corregidos mediante la revisión del código, lo que permitió obtener un programa funcional.

Finalmente, se ejecutó el programa verificando que los datos ingresados fueran procesados correctamente, obteniendo como salida la información capturada, la evaluación de condiciones y el resultado de la operación realizada. Esto permitió validar el correcto funcionamiento del programa y reforzar la importancia de probar el código con diferentes valores.

## Resultado

Como resultado de la práctica, se logró desarrollar e implementar un programa funcional en lenguaje C que permite la interacción con el usuario mediante la captura, procesamiento y visualización de datos. El programa fue compilado y ejecutado correctamente después de realizar los ajustes necesarios, lo que permitió obtener resultados coherentes y acordes a lo esperado.

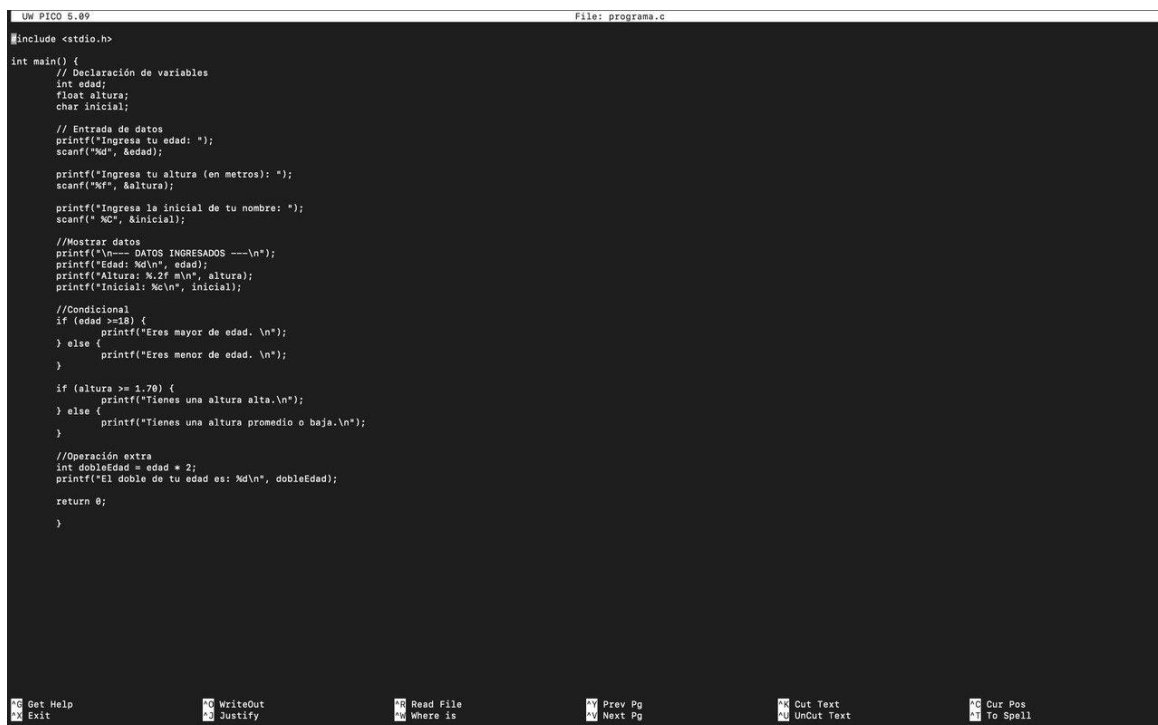
Durante la ejecución, se comprobó que los valores ingresados son almacenados adecuadamente en las variables correspondientes, respetando el tipo de dato definido. Asimismo, se observó que la información se muestra en pantalla de forma clara y organizada, aplicando formatos adecuados, lo cual facilita la interpretación de los resultados por parte del usuario.

El programa demostró un comportamiento dinámico, ya que responde de manera distinta dependiendo de los datos proporcionados. Esto permitió verificar que las condiciones implementadas funcionan correctamente, evaluando los distintos casos posibles y generando mensajes apropiados en cada situación. De esta forma, se confirmó que la lógica del programa se ejecuta de manera adecuada.

Por otro lado, el cálculo realizado dentro del programa permitió obtener nuevos valores a partir de los datos ingresados, evidenciando que no solo se limita a mostrar información, sino que también realiza un procesamiento básico de la misma. Esto refuerza la comprensión del uso de variables y operadores dentro del lenguaje C.

Durante el proceso, se identificaron errores y advertencias en la compilación, los cuales fueron corregidos mediante la revisión del código. Esto permitió comprender la importancia de cuidar la sintaxis y la correcta implementación de las funciones, así como la necesidad de realizar pruebas para garantizar el buen funcionamiento del programa.

Finalmente, al ejecutar el programa con diferentes valores de prueba, se confirmó que existe coherencia entre la entrada de datos, el procesamiento interno y la salida generada. En conjunto, estos resultados demuestran que el programa cumple con los objetivos de la práctica, validando el uso adecuado de los elementos básicos del lenguaje C y su correcta integración en un programa funcional.

A screenshot of a terminal window titled "UW PICO 5.09" and "File: programa.c". The terminal displays the source code of a C program. The code includes a header file, declares variables for age, height, and name, and contains logic for input, output, and conditional checks. The program prompts the user for age, height, and name, then displays the entered data and performs calculations based on the input. The terminal window has a dark background with light-colored text. At the bottom of the window, there is a status bar with various icons and labels such as "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", "Cur Pos", and "To Spell".

```
#include <stdio.h>

int main() {
    // Declaración de variables
    int edad;
    float altura;
    char inicial;

    // Entrada de datos
    printf("Ingresa tu edad: ");
    scanf("%d", &edad);

    printf("Ingresa tu altura (en metros): ");
    scanf("%f", &altura);

    printf("Ingresa la inicial de tu nombre: ");
    scanf(" %c", &inicial);

    //Mostrar datos
    printf("\n--- DATOS INGRESADOS ---\n");
    printf("Edad: %d\n", edad);
    printf("Altura: %.2f m\n", altura);
    printf("Inicial: %c\n", inicial);

    //Condicional
    if (edad >= 18) {
        printf("Eres mayor de edad. \n");
    } else {
        printf("Eres menor de edad. \n");
    }

    if (altura >= 1.70) {
        printf("Tienes una altura alta.\n");
    } else {
        printf("Tienes una altura promedio o baja.\n");
    }

    //Operación extra
    int dobleEdad = edad * 2;
    printf("El doble de tu edad es: %d\n", dobleEdad);

    return 0;
}
```

Figura 1. Código fuente del programa en lenguaje C

```
Last login: Thu Mar 26 21:46:27 on ttys001
jaque@Liza-MacBook-Air ~ % nano programa.c
jaque@Liza-MacBook-Air ~ % gcc programa.c -o programa
programa.c:17:18: warning: format specifies type 'wchar_t *' (aka 'int *') but the argument has type 'char *' [-Wformat]
17 |     scanf("%c", &inicial);
   |               ^
   |               |
   |               %c
   |               |
   |               %s
programa.c:32:21: error: expected ')'
32 |     if (altura >= 1.70 {
   |                       ^
programa.c:32:5: note: to match this '('
32 |     if (altura >= 1.70 {
   |     ^
programa.c:40:41: error: use of undeclared identifier 'dobleEdad'
40 |     printf("El doble de tu edad es: %d\n", dobleEdad);
   |                                         ^
1 warning and 2 errors generated.
jaque@Liza-MacBook-Air ~ % nano programa.c
jaque@Liza-MacBook-Air ~ % gcc programa.c -o programa
programa.c:17:18: warning: format specifies type 'wchar_t *' (aka 'int *') but the argument has type 'char *' [-Wformat]
17 |     scanf("%c", &inicial);
   |               ^
   |               |
   |               %c
   |               |
   |               %s
1 warning generated.
jaque@Liza-MacBook-Air ~ % ./programa
Ingresar tu edad: 19
Ingresar tu altura (en metros): 1.61
Ingresar la inicial de tu nombre: L

--- DATOS INGRESADOS ---
Edad: 19
Altura: 0.68 m
Inicial: L
Eres mayor de edad.
Tienes una altura promedio o baja.
El doble de tu edad es: 38
jaque@Liza-MacBook-Air ~ %
```

Figura 2. Proceso de compilación del programa

## Conclusión

**Díaz Rodríguez José Pablo:** Durante la práctica utilizamos programas en lenguaje C, al igual de que usamos comandos básicos para hacer que el programa actúe bien y tengamos los conocimientos básicos en lenguaje C, sin embargo durante la práctica tuve algunos problemas y por ende no tome bien las capturas, afortunadamente lo pude acabar en casa correctamente usando otra vez los comandos y volver a abrir de nuevo los programas vi y nano, completando y rectificando mis conocimientos aprendidos durante la clase de laboratorio.

**Nolasco Ramírez Sofia:** En esta práctica se logró entender mejor cómo pasar de una idea o lógica a un programa real en lenguaje C. A diferencia del pseudocódigo, aquí ya se tuvo que cuidar la sintaxis y la forma en la que se escriben las instrucciones, lo que hizo más evidente la importancia de escribir correctamente para que el programa funcione. También se pudo ver que un programa no solo muestra datos, sino que puede procesarlos, tomar decisiones con base en condiciones y generar nuevos resultados. Esto ayudó a comprender mejor cómo se usan las variables, las operaciones y las estructuras condicionales dentro de un programa.

Durante el proceso, los errores fueron parte importante del aprendizaje, ya que al corregirlos se entiende mejor cómo funciona el lenguaje y qué se debe mejorar en el código. En lo personal, esto ayudó a tener más cuidado al escribir el programa y a revisar mejor cada parte antes de compilar. Además, al probar el programa con diferentes datos, se pudo comprobar que funciona correctamente en distintos casos. En general, la práctica ayudó a reforzar las bases de programación en C y dejó claro que tener una buena lógica facilita mucho escribir el código, además de que es importante poner atención a los detalles para que todo funcione bien.

**Cruz Gómez Lizette Jacqueline:**

**Macay Martínez David:** A través del desarrollo de esta práctica, se logró utilizar en un solo programa las funciones básicas del lenguaje C: edición, compilación y ejecución. La interacción con variables de distintos tipos (enteros, flotantes, caracteres) permitió comprender cómo el lenguaje gestiona la memoria y la importancia de la precisión en la declaración de tipos.

Se validó la utilidad de las funciones de la biblioteca estándar, específicamente `printf()` y `scanf()`. Las cuales fueron importantes para la entrada y la salida de datos.

En esta práctica el uso de expresiones relacionales y lógicas fue importante para entender cómo el lenguaje evalúa condiciones de verdad. De mismo modo, se observó que el resultado de estas evaluaciones se traduce en valores numéricos (generalmente 0 para falso y 1 para verdadero), lo cual sienta las bases para el uso posterior de estructuras de control más complejas como `if` o `while`.

## **Bibliografía**

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.